基于 Pytorch 的深度学习实践 (四)

YYFL

March 2026

1 Transformer 总览

Transformer 是一种基于注意力机制（Attention）的序列建模架构。与 RNN 按时间步串行处理序列不同，Transformer 能并行处理整个序列，因此训练效率更高，也更容易建模长距离依赖。

Transformer 的核心模块包括：

- Self-Attention（自注意力）
- Multi-Head Attention（多头注意力）
- Positional Encoding（位置编码）
- Encoder-Decoder 结构
- Masked Self-Attention 与 Cross-Attention
- Add & Norm（残差连接与层归一化）
- Feed Forward Network（前馈网络）

2 Self-Attention（自注意力）

2.1 为什么需要 Self-Attention

考虑句子：

```
The animal didn't cross the street because it was too tired.
```

当模型处理单词 `it` 时，需要判断它指代的是 `animal` 还是 `street`。Self-Attention 的作用，就是让当前位置的 token 与序列中所有 token 建立关联，并根据关联强弱聚合信息。

2.2 核心思想

对于序列中的每一个 token，计算它与其他所有 token 的相关性分数，再对这些分数做归一化，最后加权求和得到新的表示。

例子： 设输入句子为：

$$[\texttt{I},\ \texttt{love},\ \texttt{deep},\ \texttt{learning}]$$

当模型更新单词 `love` 的表示时，它会计算 `love` 与句中所有单词的注意力权重。假设得到的权重分别为：

$$\alpha_{\texttt{love,I}} = 0.4, \qquad \alpha_{\texttt{love,love}} = 0.3, \qquad \alpha_{\texttt{love,deep}} = 0.2, \qquad \alpha_{\texttt{love,learning}} = 0.1$$

则 `love` 的新表示是所有词向量的加权和：

$$\mathbf{h}'_{\texttt{love}} = 0.4\,\mathbf{h}_{\texttt{I}} + 0.3\,\mathbf{h}_{\texttt{love}} + 0.2\,\mathbf{h}_{\texttt{deep}} + 0.1\,\mathbf{h}_{\texttt{learning}}$$

这说明：在更新 `love` 的表示时，模型不仅保留了它自身的信息，还融合了上下文中其他单词的信息。其中，`I` 对 `love` 的影响最大，而 `learning` 的影响较小。

2.3 Q、K、V 的含义

在 Self-Attention 中，每个 token 会被映射成三个向量：

向量	名称	类比	作用
Q	Query	搜索请求	我在找什么信息
K	Key	标签	我有哪些特征
V	Value	内容	我实际携带的信息

维度要求如下：

向量	维度要求	原因
Q 与 K	必须相同	因为要做点积 $QK^\top$
V	可以不同	但通常设置为相同，便于实现

若输入序列表示为 $X \in \mathbb{R}^{n \times d_{\text{model}}}$，则：

$$Q = XW^Q, \qquad K = XW^K, \qquad V = XW^V$$

其中：

$$W^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

2.4 缩放点积注意力

Self-Attention 的标准公式为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

这里分三步理解：

1. 计算相似度分数：$QK^\top$
2. 进行缩放：除以 $\sqrt{d_k}$，防止点积值过大导致 softmax 梯度过小
3. 对 V 做加权求和，得到新的 token 表示

2.5 为什么要缩放

当 d_k 很大时，点积的数值会变大，softmax 输出容易极端化，使梯度变得很小，不利于训练。因此使用：

$$\frac{QK^\top}{\sqrt{d_k}}$$

来稳定训练过程。

2.6 PyTorch 示例代码

```
import torch
import torch.nn as nn

x = torch.tensor([
    [1.0, 0.0, 0.0, 0.0],  # token 1
    [0.0, 1.0, 0.0, 0.0],  # token 2
    [0.0, 0.0, 1.0, 0.0],  # token 3
], dtype=torch.float32)

W_q = nn.Linear(4, 4, bias=False)
W_k = nn.Linear(4, 4, bias=False)
W_v = nn.Linear(4, 4, bias=False)

Q = W_q(x)   # (3, 4)
K = W_k(x)   # (3, 4)
V = W_v(x)   # (3, 4)

scores = torch.matmul(Q, K.T) / torch.sqrt(torch.tensor(K.size(-1), dtype=torch.
    float32))
attention_weights = torch.softmax(scores, dim=-1)
output = torch.matmul(attention_weights, V)

print("attention weights:")
print(attention_weights)
'''
注意力权重:
tensor([[0.5, 0.3, 0.2],   # "I" 关注谁
        [0.4, 0.4, 0.2],   # "love" 关注谁
        [0.3, 0.3, 0.4]])  # "deep" 关注谁
        '''
print("output:")
print(output)
```

3 Multi-Head Attention（多头注意力）

3.1 为什么需要多头

单个注意力头只能学习一种关系模式，例如：

- 某个头关注语法关系
- 某个头关注语义关系
- 某个头关注位置关系

为了同时捕捉多种模式，Transformer 使用多头注意力。

3.2 基本思想

将 Q, K, V 分别映射到多个子空间，在每个子空间中独立执行注意力计算，最后将所有头的结果拼接并再次线性变换。

$$Q_i = XW_i^Q, \qquad K_i = XW_i^K, \qquad V_i = XW_i^V$$

第 i 个头的输出为：

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$$

最终：

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

其中 h 是头数。

3.3 维度变化

设：

$$d_{\text{model}} = h \cdot d_{\text{head}}$$

输入张量形状为：

$$(\text{batch}, \text{seq_len}, d_{\text{model}})$$

切分多头后变为：

$$(\text{batch}, h, \text{seq_len}, d_{\text{head}})$$

每个头分别计算注意力，最后再拼接回：

$$(\text{batch}, \text{seq_len}, d_{\text{model}})$$

3.4 PyTorch 示例代码

```
#input:(batch,seq_len,d_model)
#经过三路线性变换得到QKV
#切成多头：把最后一位d_model拆分成num_heads*d_head
对于每一个head：(batch,seq_len,d_head)，进过自注意力得到注意力权重
```

```python
import torch
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model=512, num_heads=8):
        super().__init__()
        self.d_model=d_model
        self.num_heads=num_heads
        self.d_head=d_model//num_heads=8   #向下取整
        self.W_q=nn.Linear(d_model,d_model)
        self.W_k=nn.Linear(d_model,d_model)
        self.W_v=nn.Linear(d_model,d_model)
        self.W_o=nn.Linear(d_model,d_model)

    def forward(self,x):
        batch,seq_len,d_model=x.shape
        Q = self.W_q(x).view(batch, seq_len, self.num_heads, self.d_head).transpose
            (1, 2)
        K = self.W_k(x).view(batch, seq_len, self.num_heads, self.d_head).transpose
            (1, 2)
        V = self.W_v(x).view(batch, seq_len, self.num_heads, self.d_head).transpose
            (1, 2)
        #.view() 是PyTorch的张量重塑操作，它不改变数据，只改变视图
        #.transpose(1, 2) 交换张量的第1维（序列长度）和第2维（头数），将头维度移到前
            面，便于后续的矩阵运算
        #(batch_size, num_heads, seq_len, d_head)
        scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(
            self.d_head))
        attention_weights = torch.softmax(scores, dim=-1)
        output = torch.matmul(attention_weights, V)
        '''
        torch.matmul(A, B):
                如果A和B都是2D矩阵（形状(M, N)和(N, P)），结果是标准矩阵乘法：(M, P
                    )。
                对于高维（>2D），PyTorch将前面的维度视为“批次维度”（batch
                    dimensions），最后两个维度视为矩阵维度。
                对于每个batch和head，独立计算QK
        '''
        #concat
        output = output.transpose(1, 2).contiguous().view(batch_size, seq_len,
            d_model)
        '''
        transpose 之后，张量的内存布局是“非连续的”，它只是改变了视图，底层数据还
            是原来的顺序。
                .contiguous() 会真正复制一份连续内存的数据。为什么必须加？view() 操
                    作要求张量在内存中是连续的，否则会报错：
        '''
        return self.W_o(output)     #让模型学习如何融合不同头的不同“视角”
```

4 Positional Encoding（位置编码）

4.1 为什么需要位置编码

Self-Attention 本身只关心 token 与 token 之间的关系，不知道它们在序列中的先后顺序。例如：

猫追老鼠
老鼠追猫

这两个句子包含同样的词，但顺序不同，语义完全不同。因此必须显式加入位置信息。

4.2 基本做法

将词嵌入与位置编码逐元素相加：

$$\text{InputEmbedding} = \text{TokenEmbedding} + \text{PositionalEncoding}$$

注意：这里是同维度向量相加，而不是拼接。

4.3 正弦余弦位置编码

原始 Transformer 使用固定的位置编码：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

其中：

- pos 表示位置
- i 表示维度索引

这种设计的优点：

- 不需要训练额外参数
- 可以外推到比训练时更长的序列
- 不同维度使用不同频率，能表达相对位置信息

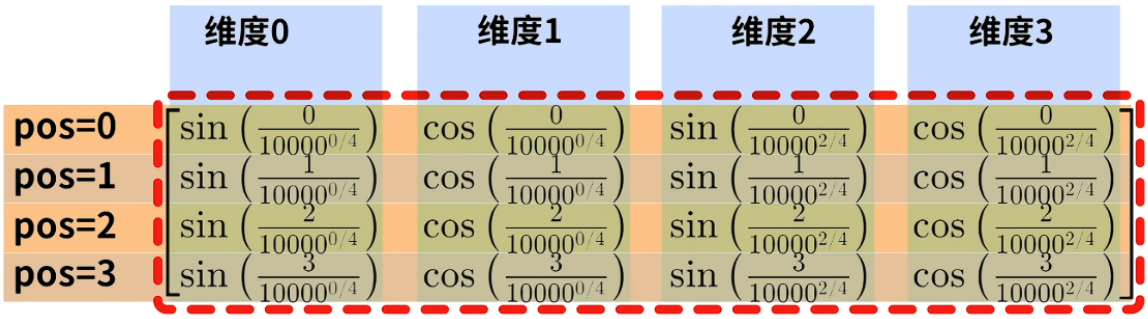

图 1: 正余弦位置编码示意图

4.4 固定位置编码与可学习位置编码对比

方式	优点	缺点
固定（sin/cos）	无需训练参数，可外推到更长序列	不能针对具体任务自适应优化
可学习位置编码	能根据任务学习更合适的位置表示	通常不能很好泛化到超过训练长度的序列

4.5 RoPE（Rotary Position Embedding）

近年来很多大语言模型使用旋转位置编码（RoPE）。RoPE 的核心思想是：不直接把位置编码“加到”输入向量上，而是把位置信息编码进 Q 和 K 的旋转变换中。

对于二维向量对，可以理解为：

$$\begin{bmatrix} x_1' \\ x_2' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

其中角度 θ 与位置相关。RoPE 的好处是：

- 更自然地建模相对位置
- 特别适合注意力中的相似度计算
- 在长上下文建模中表现较好

4.6 为什么“词向量 + 位置编码”不会混掉

虽然两者是直接相加，但它们的统计结构不同：

- 词向量主要表示语义信息
- 位置编码主要表示位置信息

后续线性层、注意力层和前馈层可以学习如何把这两类信息重新分离和利用。因此，相加并不意味着信息丢失，而是信息叠加。

5 Encoder-Decoder 结构

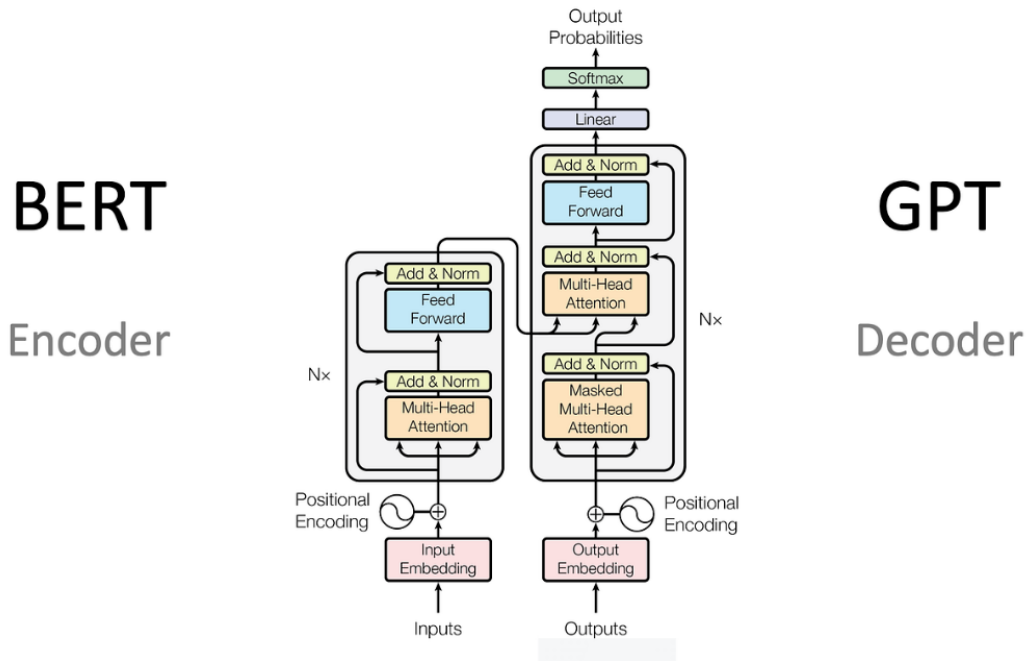

图 2: Transformer 结构示意图

5.1 三类 Transformer 架构

架构	组成	典型应用
Encoder-only	只有编码器	BERT、文本分类、抽取任务
Decoder-only	只有解码器	GPT、文本生成
Encoder-Decoder	编码器 + 解码器	翻译、摘要、问答

5.2 Encoder 的作用

Encoder 的作用是对输入序列进行上下文化表示，输出通常称为 memory。它可以看到整个输入序列，没有因果遮挡。

每层 Encoder 通常包含：

1. Multi-Head Self-Attention
2. Add & Norm
3. Feed Forward
4. Add & Norm

5.3 Decoder 的作用

Decoder 负责自回归生成。它不仅要利用已经生成的历史 token，还要利用 Encoder 输出的 memory。

每层 Decoder 通常包含：

1. Masked Multi-Head Self-Attention
2. Add & Norm
3. Cross-Attention
4. Add & Norm
5. Feed Forward
6. Add & Norm

6 Masked Self-Attention（掩码自注意力）

6.1 为什么需要 Mask

在训练时，Decoder 的输入往往是整个目标序列（Teacher Forcing）。如果不加 mask，当前位置就能“偷看未来答案”，训练会失真。

例如输入：

$$[\texttt{<BOS>}, \texttt{I}, \texttt{love}, \texttt{you}, \texttt{<EOS>}]$$

希望模型预测下一个 token 时，位置 `love` 不应该看到 `you`。

6.2 下三角 mask

Mask 通常是下三角矩阵：

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

这表示第 t 个位置只能关注自己以及之前的位置。

6.3 训练与推理时的区别

- 训练时：目标序列一次性输入，必须用 mask 防止泄露未来信息
- 推理时：模型按步生成，理论上本来就看不到未来 token

但实际实现中，推理时仍经常保留 mask，原因包括：

- 训练和推理逻辑统一
- 批量推理时仍需屏蔽 padding
- 保持模型行为与训练阶段一致

7 Cross-Attention（交叉注意力）

Cross-Attention 用于 Decoder 从 Encoder 的输出中检索信息。
其核心是：

$$Q \text{ 来自 Decoder}, \qquad K, V \text{ 来自 Encoder}$$

因此公式为：

$$\text{CrossAttention}(Q_{\text{dec}}, K_{\text{enc}}, V_{\text{enc}}) = \text{softmax}\left(\frac{Q_{\text{dec}} K_{\text{enc}}^{\top}}{\sqrt{d_k}}\right) V_{\text{enc}}$$

这使得 Decoder 在生成目标语言时，能够对齐并关注源语言中的相关部分。

8 Add & Norm（残差连接与层归一化）

8.1 残差连接（Residual Connection）

残差连接的形式为：

$$\text{Output} = x + \text{Sublayer}(x)$$

作用：

- 保留原始输入信息
- 缓解深层网络中的梯度消失问题
- 提高训练稳定性

8.2 层归一化（LayerNorm）

LayerNorm 对每个样本、每个位置的特征维度做归一化：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中：

- μ 是该位置特征向量的均值
- σ^2 是该位置特征向量的方差
- γ, β 是可学习参数

8.3 LayerNorm 与 BatchNorm 的区别

特性	BatchNorm	LayerNorm
归一化维度	跨 batch	跨特征维度
适用场景	CNN	Transformer、RNN
是否依赖 batch 大小	是	否

Transformer 更适合用 LayerNorm，因为序列长度可变，而且 batch size 可能较小。

8.4 Pre-LN 与 Post-LN

Post-LN（原始论文）

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

Pre-LN（现代实现更常见）

$$\text{Output} = x + \text{Sublayer}(\text{LayerNorm}(x))$$

一般来说，Pre-LN 在深层模型中训练更稳定。

8.5 PyTorch 示例代码

```
import torch
import torch.nn as nn

class AddNorm(nn.Module):
    def __init__(self, embed_size, dropout=0.1):
        super().__init__()
        self.norm = nn.LayerNorm(embed_size)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, sublayer):
        residual = x
        x = self.norm(x)        # Pre-LN
        x = sublayer(x)
        x = self.dropout(x)
        return x + residual

    def forward_post_ln(self, x, sublayer):
        residual = x
        x = sublayer(x)
        x = self.dropout(x)
        return self.norm(x + residual)
```

9 Feed Forward Network（前馈网络）

9.1 结构

Transformer 中的 FFN 是逐位置独立应用的两层 MLP：

$$\text{FFN}(x) = W_2\sigma(W_1x + b_1) + b_2$$

通常：

- 第一层升维：$d_{\text{model}} \to d_{ff}$
- 激活函数：ReLU 或 GELU
- 第二层降维：$d_{ff} \to d_{\text{model}}$

例如在原始 Transformer 中：

$$d_{\text{model}} = 512, \qquad d_{ff} = 2048$$

9.2 作用

FFN 的作用是：

- 提供非线性变换能力
- 对每个位置的表示进行进一步特征提取
- 补充注意力层主要负责“信息交互”的不足

9.3 PyTorch 示例代码

```
import torch
import torch.nn as nn

class FeedForward(nn.Module):
    def __init__(self, d_model, d_ff=2048, dropout=0.1):
        super().__init__()
        self.linear1 = nn.Linear(d_model, d_ff)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)
        self.linear2 = nn.Linear(d_ff, d_model)

    def forward(self, x):
        x = self.linear1(x)   # 升维
        x = self.relu(x)
        x = self.dropout(x)
        x = self.linear2(x)   # 降维
        return x
```

10 一个完整 Transformer Block 的理解

以 Encoder Block 为例，其计算流程可写为：

$$x_1 = x + \text{MHA}(\text{LN}(x))$$

$$x_2 = x_1 + \text{FFN}(\text{LN}(x_1))$$

这里：

- MHA 负责建模 token 间关系
- FFN 负责逐位置非线性变换
- 残差连接保证梯度传播和信息保留
- LayerNorm 提高训练稳定性

11 总结

Transformer 的关键思想可以概括为：

1. 用 Self-Attention 建模任意位置间的依赖关系
2. 用 Multi-Head 让模型从多个子空间学习不同关系模式
3. 用位置编码补足顺序信息
4. 用 Mask 实现自回归约束
5. 用 Cross-Attention 连接输入与输出序列
6. 用 Add & Norm 与 FFN 提高训练稳定性和表达能力

因此，Transformer 本质上是一个“注意力驱动的序列表示与生成框架”。

12 附录

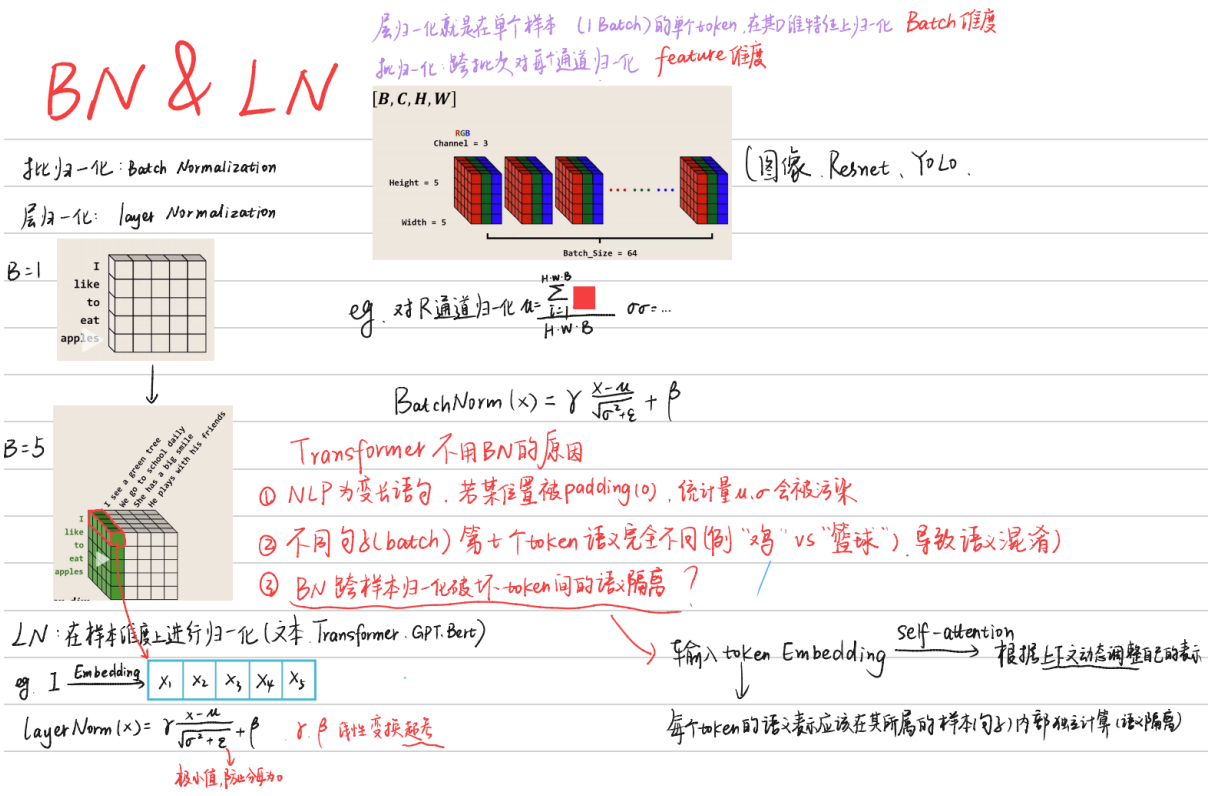

图 3: BN LN 手写笔记